

クラス	学籍番号	氏名
Cプログラミング 入門（総機3）		パターン A

次のコードの実行結果（出力）を書け。

<pre>#include <stdio.h> int main() { puts("100 + 23"); printf("%03d%03d\n", 1, 2345); printf("%c %d\n", 63, 'A'); }</pre>	<pre>#include <stdio.h> bool A() { puts("A"); return true; } bool B() { puts("B"); return false; } int main() {</pre>
--	--

[illegible]

```
#include <stdio.h>
#include <ctype.h>

int main()
{
    for (int i = 32; i < 64; i += 2)
    {
        if (isdigit(i))
        {
            putchar(i);
        }
    }
}
```

<pre>#include <stdio.h> void Show(int n, char c) { for (int i = 0; i < n; ++i) { putchar(c); } putchar('\n'); } int main() { for (int i = 0; i < 4; ++i) { Show((i + 1), ('A' + i)); } }</pre>	<pre>#include <stdio.h> typedef enum { RED, BLUE = 100, YELLOW, PINK = 120, GREEN, } Color; int main() { printf("%d\n", RED); printf("%d\n", YELLOW); printf("%d\n", GREEN); }</pre>
A	0
BB	101
CCC	121
DDDD	

式を評価した結果の型名 (int, char, double, size_t など) と、結果の値を書け。

	型	値		型	値
5 / 2LL	long long	2LL	strlen("")	size_t	0
ceil(2.6)	double	3.0	'A' + 1	int	66
-3 < -2 < -1	int	0	1.2 * 0	double	0.0

次のコードを実行し、右の入力（複数行）を一度に与えた場合の出力を書け。

#include <stdio.h>	入力
int main()	49
{	50
const int c1 = getchar();	51
const int c2 = getchar();	
const int c3 = getchar();	
printf("%d\n", (c1 + c2 + c3));	119
printf("%c\n", (c1 + c2));	
}	m

#include <stdio.h>	入力
int main()	48
{	50
int a, b;	
scanf("%d %d", &a, &b);	
while (a != b)	49
{	50
a += (a < b) ? 1 : -1;	
printf("%d\n", a);	
}	
}	

次の問いに答えよ。選択肢があるものは答えをすべて○で囲め。

isxdigit(c) が真になる ASCII 文字は	22 種類
講義環境での sizeof(char) の結果 (数値)	1
puts("Hello") の "Hello" は	仮引数 {実引数} どちらでもない
演算子「?:」の名前を書け	条件 (三項) 演算子
printf で double 型の値を小数点以下 2 桁で出力するための書式指定子を書け	%.2f
アドレス演算子に使う記号を書け	&
右結合である演算子をすべて選べ	< && = / % +=
講義環境で time 関数が返す、1970-01-01 00:00:00 UTC からの経過時間を一般に何と呼ぶか	Unix 時間 (エポック秒)

(参考) ASCII コード表

10 進	16 進	文字	10 進	16 進	文字	10 進	16 進	文字	10 進	16 進	文字
0	00	\0 (ヌル文字)	32	20	space	64	40	@	96	60	`
1	01	SOH	33	21	!	65	41	A	97	61	a
2	02	STX	34	22	"	66	42	B	98	62	b
3	03	ETX	35	23	#	67	43	C	99	63	c
4	04	EOT	36	24	\$	68	44	D	100	64	d
5	05	ENQ	37	25	%	69	45	E	101	65	e
6	06	ACK	38	26	&	70	46	F	102	66	f
7	07	\a (ベル)	39	27	'	71	47	G	103	67	g
8	08	\b (バックスペース)	40	28	(	72	48	H	104	68	h
9	09	\t (水平タブ)	41	29	)	73	49	I	105	69	i
10	0A	\n (改行)	42	2A	*	74	4A	J	106	6A	j
11	0B	\v (垂直タブ)	43	2B	+	75	4B	K	107	6B	k
12	0C	\f (改ページ)	44	2C	,	76	4C	L	108	6C	l
13	0D	\r (復帰)	45	2D	-	77	4D	M	109	6D	m
14	0E	SO	46	2E	.	78	4E	N	110	6E	n
15	0F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL

Cプログラミング入門のTAとして、後輩からの質問に答えよ。図やコードを用いてもよい。（配点: 質問への直接回答 3 点、理由や補足の説明（1〜2 文）2 点、計 5 点）

そもそも最初のサンプルの時点でわからないことが多すぎるんですね。
stdio.h って何ですか？
int って何ですか？

```
#include <stdio.h>

int main()
{
    puts("Apple");
}
```

stdio.h は標準入出力に関する機能を提供する標準ライブラリヘッダ。このサンプルでは puts 関数を使うためにインクルードしている。stdio は standard input / output の略。
int は integer の略で、整数型を表す。このサンプルの int main は、main 関数が最後に整数を返すことを表す。プログラムの正常終了時、main 関数は 0 を返す。

このサンプルでは、ターミナルから「AA」や「ab」みたいに 2 文字入力するのはわかったんですけど、間違って「ABC」とか「abc」みたいに 3 文字以上入力したらまずいんですか？

```
#include <stdio.h>

int main()
{
    int a = getchar();
    int b = getchar();
    printf("%c %c", a, b);
}
```

getchar() は1回につき1文字だけ読み取るため、最初の2文字だけが a と b に格納される。三文字目以降は標準入力に残る。このプログラムはそのまま終了し、標準入力に残った内容は使われずに消去される。もし、さらに入力を行うコード（getchar など）を書いていると、残った文字が次の入力として読み込まれる。

乱数のシードを毎回異なるものにすれば、生成される乱数のパターンも変わるっていうからこうしたんですけど、どうでしょう？

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main()
{
    for (int i = 0; i < 5; ++i)
    {
        srand((unsigned)time(NULL));
        printf("%d\n", rand());
    }
}
```

この位置で毎回 srand を呼ぶのは適切ではない。time(NULL) は秒単位で変化する。ループは通常 1 秒未満で終了するため、各反復で同じシードが設定される可能性が高い。同じシードを設定した直後の最初の rand() は同じ値になるため、同じ値が繰り返し出力されやすい。シードはループの前で一度だけ設定する。

次の 2 つのプログラムは同じですか？ グローバル変数と static 変数は、どう使い分ければいいんですか？

```
int g_count;

int GenerateID()
{
    int id = g_count;
    ++g_count;
    return id;
} // 実装 A
```

```
int GenerateID()
{
    static int count = 0;
    int id = count;
    ++count;
    return id;
} // 実装 B
```

どちらも呼び出すたびに 0, 1, 2, ... を返し、結果は同じである。ただし、変数を参照できる範囲が異なる。static 変数 count は GenerateID の内部でしか使用できないが、グローバル変数 g_count はほかの関数からもアクセスできる。カウンタを GenerateID 内だけで使いたいなら static 変数、「ID をリセットする関数」のような別の関数とも共有するのであればグローバル変数を利用する。

このコードの s1 と s2 はどちらも同じ結果になるということではないんですか？

```
#include <stdio.h>

int main()
{
    const char s1[] = { 'A', 'p', 'p', 'l', 'e' };
    const char s2[] = "Apple";
    puts(s1);
    puts(s2);
}
```

s1 の要素数は 5 であり、文字列の終端を表すヌル文字 '\0' がない。ヌル終端されていない配列を puts 関数に渡すと未定義動作である。正しくは次のようにする。
const char s1[] = { 'A', 'p', 'p', 'l', 'e', '\0' };

格子状の盤面を使うボードゲームを考える。盤面の各マスには座標が割り当てられている。左下のマスを (0, 0) とし、右へ 1 マス進むと X 座標が 1 増え、上へ 1 マス進むと Y 座標が 1 増えるものとする。
盤面上の駒が、座標 $P_1(x1, y1)$ から $P_2(x2, y2)$ へ移動できるかどうかを、bool 型の値で返す関数が、駒ごとに定義されている。
各関数が表す駒の移動規則を読み取り、それぞれの駒について、現在位置 (▲) から 1 手で移動できるすべてのマスに○を記入せよ。

			○		
			○		
		▲			

【例】上に 1 マスまたは 2 マス移動できる駒

```
bool IsLegalMove(int x1, int y1, int x2, int y2)
{
    int dx = x2 - x1;
    int dy = y2 - y1;
    return (dx == 0) && (dy == 1 || dy == 2);
}
```

○	○	○	○	○	○
○	○	○	○	○	○
○	○	○	○	○	○
			▲		

```
bool IsLegalMove(int x1, int y1, int x2, int y2)
{
    return (y2 > y1);
}
```

			○		
		○	○	○	
	○	○	▲	○	○
		○	○		
			○		

```
bool IsLegalMove(int x1, int y1, int x2, int y2)
{
    const int dx = abs(x2 - x1);
    const int dy = abs(y2 - y1);
    const int distance = dx + dy;
    return (0 < distance) && (distance <= 2);
}
```

○						○
	○				○	
		○		○		
			▲			
		○		○		
	○				○	
○						○

```
bool IsLegalMove(int x1, int y1, int x2, int y2)
{
    const int dx = abs(x2 - x1);
    const int dy = abs(y2 - y1);
    return (dx == dy) && (dx != 0);
}
```

図書館に寄贈された本の冊数が、1 月から 12 月まで順番に与えられている。職員の大隈さんは、指定した from 月から to 月までの期間に寄贈された合計冊数を求めるプログラムを次のように作った。

```
#include <stdio.h>

int main()
{
    // 1 月は 6 冊, 2 月は 4 冊, ...
    int counts[12] = { 6, 4, 3, 7, 5, 1, 2, 3, 10, 4, 2, 7 };
    int psum[13] = {};

    for (int i = 0; i < 12; ++i)
    {
        psum[i + 1] = (psum[i] + counts[i]);
    }

    for (;;)
    {
        int from, to;
        scanf("%d %d", &from, &to);

        if ((from <= 0) || (to <= 0) || (12 < from) || (12 < to) || (to < from))
        {
            break;
        }

        printf("%d\n", (psum[to] - psum[_____]));
    }
}
```

for (;;) に入る時点での配列 psum の要素を順に書け。

0	6	10	13	20	25	26	28	31	41	45	47	54
---	---	----	----	----	----	----	----	----	----	----	----	----

コード内の下線部空欄にあてはまるコードを書け。

```
from - 1
```

psum[i] が表す値を説明せよ。また、from 月から to 月までの合計冊数を counts の要素を順に加算して求める方法と比較して、このプログラムで psum を用いる利点を説明せよ。

配列 counts の先頭から i 個分の合計、すなわち $1 \leq i$ では 1 月から i 月までの寄贈冊数の合計を表す。counts を毎回順に加算する方法では、1 回の問い合わせにつき最大 12 回程度の加算が必要になる。psum を最初に構築すれば、その後は各問い合わせを引き算 1 回 (2 回) で処理できる。問い合わせが多数になる場合に、計算量を少なくできる。

クラス	学籍番号	氏名
Cプログラミング入門（総機3）		パターン B

次のコードの実行結果（出力）を書け。

<pre>#include <stdio.h> #include <ctype.h> int main() { for (int i = 40; i <= 90; i += 10) { if (isalpha(i)) { putchar(i); } } }</pre>	FPZ
--	-----

<pre>#include <stdio.h> void Show(int n, char c) { for (int i = 0; i < n; ++i) { putchar(c); } putchar('\n'); } int main() { for (int i = 0; i < 4; ++i) { Show((i + 1), ('a' + i)); } }</pre>	<pre>#include <stdio.h> typedef enum { RED, BLUE = 200, YELLOW, PINK = 220, GREEN, } Color; int main() { printf("%d\n", RED); printf("%d\n", YELLOW); printf("%d\n", GREEN); }</pre>
--	--

a	0
bb	201
ccc	221
dddd	

<pre>#include <stdio.h> int main() { printf("%s\n", "50 + 50"); printf("%02d%02d\n", 1, 234); printf("%c %d\n", 48, '5'); }</pre>	<pre>#include <stdio.h> bool A() { puts("A"); return false; } bool B() { puts("B"); return true; } int main() { if (A() && A()) { puts("C"); } if (A() B()) { puts("D"); } }</pre>
--	---

<pre>#include <stdio.h> int main() { int a = 3, b = 4; printf("%d %d\n", a, b); { int a = 5, b = 6; printf("%d %d\n", a, b); a += 2; b += 2; } printf("%d %d\n", a, b); }</pre>	A A B D
3 4	
5 6	
3 4	

（参考）ASCII コード表

10進	16進	文字	10進	16進	文字	10進	16進	文字	10進	16進	文字
0	00	\0（ヌル文字）	32	20	space	64	40	@	96	60	`
1	01	SOH	33	21	!	65	41	A	97	61	a
2	02	STX	34	22	"	66	42	B	98	62	b
3	03	ETX	35	23	#	67	43	C	99	63	c
4	04	EOT	36	24	\$	68	44	D	100	64	d
5	05	ENQ	37	25	%	69	45	E	101	65	e
6	06	ACK	38	26	&	70	46	F	102	66	f
7	07	\a（ベル）	39	27	'	71	47	G	103	67	g
8	08	\b（バックスペース）	40	28	(	72	48	H	104	68	h
9	09	\t（水平タブ）	41	29	)	73	49	I	105	69	i
10	0A	\n（改行）	42	2A	*	74	4A	J	106	6A	j
11	0B	\v（垂直タブ）	43	2B	+	75	4B	K	107	6B	k
12	0C	\f（改ページ）	44	2C	,	76	4C	L	108	6C	l
13	0D	\r（復帰）	45	2D	-	77	4D	M	109	6D	m
14	0E	SO	46	2E	.	78	4E	N	110	6E	n
15	0F	SI	47	2F	/	79	4F	O	111	6F	o
16	10	DLE	48	30	0	80	50	P	112	70	p
17	11	DC1	49	31	1	81	51	Q	113	71	q
18	12	DC2	50	32	2	82	52	R	114	72	r
19	13	DC3	51	33	3	83	53	S	115	73	s
20	14	DC4	52	34	4	84	54	T	116	74	t
21	15	NAK	53	35	5	85	55	U	117	75	u
22	16	SYN	54	36	6	86	56	V	118	76	v
23	17	ETB	55	37	7	87	57	W	119	77	w
24	18	CAN	56	38	8	88	58	X	120	78	x
25	19	EM	57	39	9	89	59	Y	121	79	y
26	1A	SUB	58	3A	:	90	5A	Z	122	7A	z
27	1B	ESC	59	3B	;	91	5B	[	123	7B	{
28	1C	FS	60	3C	<	92	5C	\	124	7C	
29	1D	GS	61	3D	=	93	5D	]	125	7D	}
30	1E	RS	62	3E	>	94	5E	^	126	7E	~
31	1F	US	63	3F	?	95	5F	_	127	7F	DEL

式を評価した結果の型名（int, char, double, size_t など）と、結果の値を書け。

	型	値		型	値
21 / 2LL	long long	10LL	strlen("\n")	size_t	1
round(2.6)	double	3.0	'A' + 1	int	66
-3 < -2 < -1	int	0	1.2 * 0	double	0.0

次のコードを実行し、右の入力（複数行）を一度に与えた場合の出力を書け。

<pre>#include <stdio.h> int main() { const int c1 = getchar(); const int c2 = getchar(); const int c3 = getchar(); printf("%d\n", (c1 + c2 + c3)); printf("%c\n", (c1 + c2)); }</pre>	入力 49 50 51 119 m
<pre>#include <stdio.h> int main() { int a, b; scanf("%d %d", &a, &b); while (a != b) { a += (a < b) ? 1 : -1; printf("%d\n", a); } }</pre>	入力 52 50 51 50

次の問いに答えよ。選択肢があるものは答えをすべて ○ で囲め。

isalnum(c) が真になる ASCII 文字は	62 種類
講義環境での sizeof(int) の結果（数値）	4
puts("Hello") の "Hello" は	仮引数 実引数 どちらももない
演算子「?:」の名前を書け	条件 （三項） 演算子
printf で double 型の値を小数点以下 2 桁で出力するための書式指定子を書け	%.2f
間接参照演算子に使う記号を書け	*
右結合である演算子をすべて選べ	< = / % +=
関数から返ってくる値のことを一般に何と呼ぶか	戻り値 （返り値）

Cプログラミング入門のTAとして、後輩からの質問に答えよ。図やコードを用いてもよい。（配点: 質問への直接回答 3 点、理由や補足の説明（1〜2 文）2 点、計 5 点）

私が持ってる参考書では、main 関数の最後に return 0; があるんですけど、このサンプルにはありません。どちらが正しいんですか？	<pre>#include <stdio.h> int main() { puts("Apple"); }</pre>
---	--

main 関数の末尾の } に到達した場合、return 0; を実行したものととして扱われるため、どちらも同じで正しく、正常終了を表す 0 を返す。なお、これは main 関数のみの特別な規則であり、戻り値が必要な main 以外の関数で同様に省略できるわけではない。

このサンプルでは、ターミナルから「AA」や「ab」みたいに 2 文字入力するのはわかったんですけど、間違って「ABC」とか「abc」みたいに 3 文字以上入力したらまずいんですか？	<pre>#include <stdio.h> int main() { int a = getchar(); int b = getchar(); printf("%c %c", a, b); }</pre>
--	--

getchar() は1回につき1文字だけ読み取るため、最初の2文字だけが a と b に格納される。三文字目以降は標準入力に残る。このプログラムはそのまま終了し、標準入力に残った内容は使われずに消去される。もし、さらに入力を行うコード（getchar など）を書いていたれば、残った文字が次の入力として読み込まれる。

乱数のシードを毎回異なるものにすれば、生成される乱数のパターンも変わるっていうからこうしたんですけど、どうでしょう？	<pre>#include <stdio.h> #include <stdlib.h> int main() { srand(rand()); for (int i = 0; i < 5; ++i) { printf("%d\n", rand()); } }</pre>
--	---

適切なシード設定になっていない。まだ srand を呼んでいない状態の rand() は、固定シードで初期化された場合と同じ、固定パターンの乱数列を生成する。したがって、srand(rand()) で渡される値も毎回同じであり、プログラムを実行するたびに同じ乱数列になる。実行ごとに結果を変えたいのであれば、時刻をシードに使用するなどの処理が必要。

次の 2 つのプログラムは同じですか？ グローバル変数と static 変数は、どう使い分ければいいんですか？	<pre>int g_count; int GenerateID() { int id = g_count; ++g_count; return id; } // 実装 A</pre>	<pre>int GenerateID() { static int count = 0; int id = count; ++count; return id; } // 実装 B</pre>
---	---	---

どちらも呼び出すたびに 0, 1, 2, ... を返し、結果は同じである。ただし、変数を参照できる範囲が異なる。static 変数 count は GenerateID の内部でしか使用できないが、グローバル変数 g_count はほかの関数からもアクセスできる。カウンタを GenerateID 内だけで使いたいなら static 変数、「ID をリセットする関数」のような別の関数とも共有するのであればグローバル変数を利用する。

このコードの s1 と s2 はどちらも同じ結果になるということδειいんですか？	<pre>#include <stdio.h> int main() { const char s1[] = { 'A', 'p', 'p', 'l', 'e' }; const char s2[] = "Apple"; puts(s1); puts(s2); }</pre>
---	---

s1 の要素数は 5 であり、文字列の終端を表すヌル文字 '\0' がない。ヌル終端されていない配列を puts 関数に渡すと未定義動作である。正しくは次のようにする。
const char s1[] = { 'A', 'p', 'p', 'l', 'e', '\0' };

格子状の盤面を使うボードゲームを考える。盤面の各マスには座標が割り当てられている。左下のマスを (0, 0) とし、右へ 1 マス進むと X 座標が 1 増え、上へ 1 マス進むと Y 座標が 1 増えるものとする。盤面上の駒が、座標 P_1 (x1, y1) から P_2 (x2, y2) へ移動できるかどうかを、bool 型の値で返す関数が、駒ごとに定義されている。各関数が表す駒の移動規則を読み取り、それぞれの駒について、現在位置 (▲) から 1 手で移動できるすべてのマスに○を記入せよ。

	【例】上に 1 マスまたは 2 マス移動できる駒 <pre>bool IsLegalMove(int x1, int y1, int x2, int y2) { int dx = x2 - x1; int dy = y2 - y1; return (dx == 0) && (dy == 1 dy == 2); }</pre>
--	---

	<pre>bool IsLegalMove(int x1, int y1, int x2, int y2) { const int dx = x2 - x1; const int dy = y2 - y1; return ((dx == 0) != (dy == 0)); }</pre>
--	--

	<pre>bool IsLegalMove(int x1, int y1, int x2, int y2) { const int dx = abs(x2 - x1); const int dy = abs(y2 - y1); return (dx == 1 && dy == 2) (dx == 2 && dy == 1); }</pre>
--	---

	<pre>bool IsLegalMove(int x1, int y1, int x2, int y2) { const int dx = abs(x2 - x1); const int dy = abs(y2 - y1); return (dx == dy) && (dx != 0); }</pre>
--	---

図書館に寄贈された本の冊数が、1 月から 12 月まで順番に与えられている。職員の大隈さんは、指定した from 月から to 月までの期間に寄贈された合計冊数を求めるプログラムを次のように作った。

<pre>#include <stdio.h> int main() { // 1 月は 6 冊, 2 月は 4 冊, ... int counts[12] = { 6, 4, 3, 7, 5, 1, 2, 3, 10, 4, 2, 3 }; int psum[13] = {}; for (int i = 0; i < 12; ++i) { psum[i + 1] = (psum[i] + counts[i]); } for (;;) { int from, to; scanf("%d %d", &from, &to); if ((from <= 0) (to <= 0) (12 < from) (12 < to) (to < from)) { break; } printf("%d\n", (psum[to] - psum[_____])); } }</pre>
--

for (;;) に入る時点での配列 psum の要素を順に書け。

0	6	10	13	20	25	26	28	31	41	45	47	50
---	---	----	----	----	----	----	----	----	----	----	----	----

コード内の下線部空欄にあてはまるコードを書け。
from - 1

psum[i] が表す値を説明せよ。また、from 月から to 月までの合計冊数を counts の要素を順に加算して求める方法と比較して、このプログラムで psum を用いる利点を説明せよ。

配列 counts の先頭から i 個分の合計、すなわち 1 ≤ i では 1 月から i 月までの寄贈冊数の合計を表す。counts を毎回順に加算する方法では、1 回の問い合わせにつき最大 12 回程度の加算が必要になる。psum を最初に構築すれば、その後は各問い合わせを引き算 1 回 (2 回) で処理できる。問い合わせが多数になる場合に、計算量を少なくできる。